
بسم الله الرحمن الرحيم

موضوع : Merge Conflict

نام ارائه دهنده: فاطمه جعفری،فاطمه قنبری

نام استاد :سمیه علی اکبری

تاریخ ارائه :۱۴۰۵/۰۷/۲۶



Git چیست؟

گیت یک سیستم کنترل نسخه است که برای ثبت، مدیریت و پیگیری تغییرات فایل‌ها و کدهای برنامه‌نویسی استفاده می‌شود. این ابزار به برنامه‌نویسان کمک می‌کند تاریخچه تغییرات پروژه را مشاهده کنند، نسخه‌های قبلی را بررسی کنند و بدون ایجاد تغییر مستقیم در نسخه اصلی، قابلیت‌های جدیدی به پروژه اضافه کنند.

مهم ترین کاربرد های git:

- ثبت تغییرات پروژه
- مشاهده تاریخچه تغییرات
- بازگشت به نسخه‌های قبلی
- ایجاد شاخه‌های جداگانه برای توسعه
- ادغام تغییرات بخش‌های مختلف پروژه

GIT HUB چیست؟

گیت‌هاب یک پلتفرم آنلاین برای نگهداری و مدیریت پروژه‌هایی است که با گیت کنترل نسخه می‌شوند. این پلتفرم به برنامه‌نویسان اجازه می‌دهد پروژه‌های خود را به اشتراک بگذارند و به‌صورت گروهی روی آن‌ها کار کنند.

مهم ترین کاربرد های git hub:

- ذخیره آنلاین پروژه‌ها
- همکاری گروهی برنامه‌نویسان
- بررسی کدهای نوشته‌شده
- مدیریت مشکلات و وظایف پروژه
- بررسی و ادغام تغییرات برنامه‌نویسان

تفاوت git و github:

گیت ابزاری برای مدیریت نسخه‌ها و ثبت تغییرات پروژه است، درحالی‌که گیت‌هاب بستری آنلاین برای میزبانی پروژه‌ها و همکاری میان برنامه‌نویسان فراهم می‌کند. برای مثال، برنامه‌نویس می‌تواند تغییرات پروژه را با گیت روی کامپیوتر خود ثبت کند و سپس با استفاده از گیت‌هاب آن را با سایر اعضای تیم به اشتراک بگذارد.

Merge چیست؟

مرج به معنای ادغام تغییرات دو شاخه از یک پروژه است .
این قابلیت در گیت به برنامه نویسان کمک می کند تغییراتی را که به صورت جداگانه روی بخش های مختلف پروژه ایجاد کرده اند،با یکدیگر ترکیب کنند.

برای مثال :

فرض کنید دو برنامه نویس روی یک وب سایت کار می کنند .
یک نفر صفحه ورود کاربران را طراحی می کند و نفر دیگر صفحه ثبت نام را توسعه می هد.
با استفاده از مرج می توان تغییرات هر دو بخش را در پروژه اصلی ترکیب کرد.

هدف از merge:



- ترکیب تغییرات شاخه های مختلف
- همکاری راحت تر اعضای تیم روی یک پروژه
- حفظ تاریخچه تغییرات پروژه
- انتقال قابلیت های جدید به شاخه اصلی پروژه
- یکپارچه سازی کدهای نوشته شده توسط افراد مختلف

شاخه چیست و چه ارتباطی با mergeدارد؟

در گیت، به هر مسیر مستقل برای توسعه پروژه، یک شاخه گفته می‌شود.
شاخه به برنامه‌نویس اجازه می‌دهد بدون تغییر مستقیم در نسخه اصلی پروژه، روی یک قابلیت جدید یا اصلاح یک مشکل کار کند.

برای مثال، یک پروژه می‌تواند سه شاخه داشته باشد:

- شاخه اصلی برای نگهداری نسخه اصلی پروژه
- شاخه ورود کاربران برای طراحی صفحه ورود
- شاخه ثبت‌نام برای ساخت صفحه ثبت‌نام

پس از تکمیل کارها، می‌توان شاخه‌های ورود و ثبت‌نام را با استفاده از مرج در شاخه اصلی ادغام کرد.

نکته مهم: شاخه‌ها باعث می‌شوند برنامه‌نویسان بتوانند هم‌زمان روی بخش‌های مختلف یک پروژه کار کنند و بعد تغییراتشان را با هم ترکیب کنند.

Merge چگونه انجام می شود؟

مرحله اول:تغییرات مورد نظررا در یک شاخه جداگانه ایجاد می کنیم.

مرحله دوم:تغییرات را ثبت می کنیم تا در تاریخچه پروژه ذخیره شوند.

مرحله سوم:به شاخه ای می رویم که قرار است تغییرات در ان قرار بگیرند.

مرحله چهارم:دستور مرج را اجرا می کنیم تا گیت تغییرات را ادغام کند.

مثال عملی:

فرض کنید شاخه‌ای به نام feature داریم و می‌خواهیم تغییرات آن را وارد شاخه اصلی کنیم.

ابتدا به شاخه اصلی می‌رویم:

```
git switch main
```

سپس دستور ادغام را اجرا می‌کنیم:

```
git merge feature
```

با اجرای دستور دوم، گیت تلاش می‌کند تغییرات شاخه featureرا در شاخه فعلی، یعنی main، ادغام کند.

Merge در گیت هاب چگونه انجام می شود؟

در گیت‌هاب، معمولاً از قابلیتی به نام پول ریکوئست برای پیشنهاد و بررسی تغییرات استفاده می‌شود.
فرض کنید برنامه‌نویسی یک قابلیت جدید به پروژه اضافه کرده است. او می‌تواند تغییرات خود را در گیت‌هاب قرار دهد و درخواست کند که این تغییرات به شاخه اصلی اضافه شوند.

مراحل کار به این صورت است:

۱. برنامه‌نویس تغییرات خود را در گیت‌هاب قرار می‌دهد.
۲. یک درخواست ادغام ایجاد می‌کند.
۳. اعضای تیم کدها را بررسی می‌کنند.
۴. در صورت تأیید، تغییرات در شاخه مقصد ادغام می‌شوند.

تفاوت مهم: پول ریکوئست برای پیشنهاد و بررسی تغییرات است، اما مرج عملیات ادغام آن تغییرات است.

:Merge conflict

گاهی اوقات دو برنامه‌نویس یک قسمت از فایل را به شکل‌های متفاوت تغییر می‌دهند. در این شرایط، گیت ممکن است نتواند تشخیص دهد کدام تغییر باید باقی بماند.
به این وضعیت، تعارض در ادغام گفته می‌شود.

مثال ساده فرض کنید برنامه‌نویس اول این متن را نوشته است:

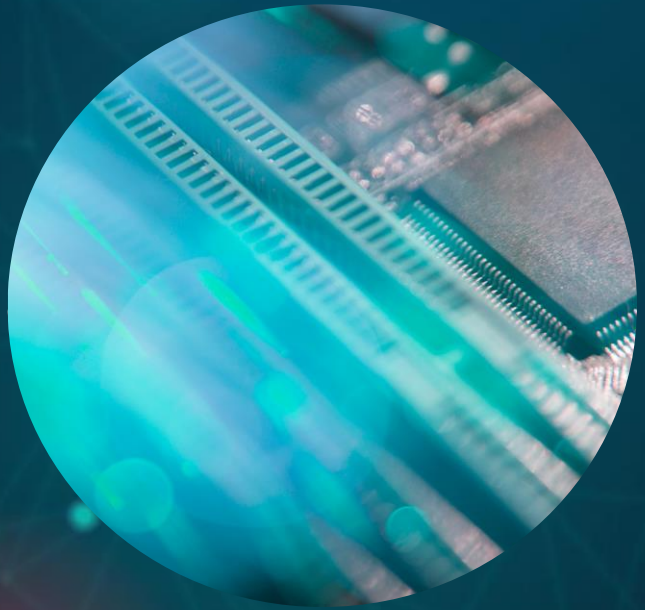
```
message = "Hello"
```

اما برنامه‌نویس دوم همان خط را به شکل زیر تغییر داده است:

```
message = "Welcome"
```

اگر این دو تغییر با هم تعارض داشته باشند، گیت از برنامه‌نویس می‌خواهد نسخه نهایی را مشخص کند.

انواع merge conflict:



تعارض‌ها در گیت ممکنه به شکل‌های مختلفی ایجاد بشوند:

۱. تعارض در یک خط از کد
وقتی دو نفر یک خط از یک فایل رو به شکل متفاوت تغییر می‌دهند، ممکنه گیت نتواند تصمیم بگیرد کدوم تغییر را نگه دارد.

۲. تعارض در حذف و ویرایش
وقتی یک نفر فایلی رو حذف می‌کند و نفر دیگه همون فایل رو تغییر می‌دهد، ممکنه تعارض ایجاد بشود.

۳. تعارض در تغییرات همزمان
وقتی تغییرات دو شاخه با هم سازگار نباشند، ممکنه ادغام خودکار انجام نشود.

چگونه تعارض را حل می کنیم؟

- فایل دارای تعارض را پیدا می‌کنیم.
- تغییرات هر دو شاخه را بررسی می‌کنیم.
- نسخه مناسب را انتخاب می‌کنیم یا تغییرات را با هم ترکیب می‌کنیم.
- فایل را ذخیره می‌کنیم و نتیجه را ثبت می‌کنیم.
- نکته: بعد از حل تعارض باید مطمئن شویم که برنامه به‌درستی کار می‌کند.

با تشکر از توجه شما